



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Προχωρημένα Θέματα Κυβερνοασφάλειας και Τεχνητής Νοημοσύνης

Hybrid Intrusion Detection with Snort and Machine Learning

Επιβλέποντες Καθηγητές: Χρήστος Ξενάκης, Αποστόλης Ζάρρας και
Γεώργιος Βούρος

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
05/07/2026

Εισαγωγή

Κάθε δίκτυο ενός οργανισμού δέχεται καθημερινά προσπάθειες παραβίασης, από αυτοματοποιημένες σαρώσεις έως στοχευμένες επιθέσεις. Επειδή ο όγκος της κίνησης είναι τεράστιος και οι επιθέσεις εκδηλώνονται σε ταχύτητες μηχανής, ο χειροκίνητος έλεγχος είναι πρακτικά αδύνατος. Τη λειτουργία αυτή αναλαμβάνει ένα σύστημα ανίχνευσης εισβολών (Intrusion Detection System), δηλαδή ένας αυτοματοποιημένος μηχανισμός που παρακολουθεί την κίνηση ενός δικτύου και ειδοποιεί όταν διακρίνει κάτι επιθετικό. Η παρακολούθηση αυτή αποτελεί στην ουσία ένα πρόβλημα δεδομένων, καθώς πρέπει να ξεχωρίσει η ελάχιστη κακόβουλη δραστηριότητα μέσα σε έναν τεράστιο όγκο φυσιολογικής κίνησης, αντιμετωπίζοντας τις κλασικές προκλήσεις του όγκου, της ταχύτητας και του θορύβου (διάλεξη 1).

Αντικείμενο της εργασίας δεν είναι απλώς η επίδειξη ότι ένα τέτοιο σύστημα ανιχνεύει επιθέσεις, αλλά η συστηματική μέτρηση του πόσο καλά το κάνει, με ποιον τρόπο και με ποια όρια. Για τον σκοπό αυτό στήθηκε ένα ελεγχόμενο εργαστηριακό περιβάλλον, παρήχθη ρεαλιστική νόμιμη και κακόβουλη κίνηση, εφαρμόστηκαν δύο διαφορετικές τεχνολογίες ανίχνευσης και εν συνεχεία συνδυάστηκαν σε ένα ενιαίο υβριδικό σύστημα.

Υπάρχουν δύο μεγάλες οικογένειες τεχνικών ανίχνευσης, και ολόκληρη η εργασία στηρίζεται στη διαφορά τους. Η πρώτη ονομάζεται ανίχνευση βάσει υπογραφών (signature-based). Λειτουργεί όπως ένα κλασικό antivirus, το οποίο αναγνωρίζει έναν ιό επειδή γνωρίζει εκ των προτέρων το αποτύπωμά του. Στηρίζεται σε ρητούς κανόνες της μορφής, εφόσον εντοπιστεί ένα συγκεκριμένο μοτίβο, σημαίνεται συναγερμός. Είναι πολύ ακριβής, καθώς δεν παράγει συναγερμό για κίνηση που δεν ταιριάζει σε γνωστό μοτίβο, έχει όμως μια θεμελιώδη αδυναμία, την οποία τονίζει και η θεωρία (διάλεξη 4), δεν εντοπίζει ό,τι δεν έχει ξαναδεί, ούτε επιθέσεις που κρύβονται επίτηδες.

Η δεύτερη οικογένεια ονομάζεται ανίχνευση βάσει ανωμαλιών ή μηχανικής μάθησης. Αντί για ρητούς κανόνες, το σύστημα μαθαίνει από παραδείγματα πώς μοιάζει η φυσιολογική κίνηση και σημαίνει ό,τι αποκλίνει από αυτήν. Το πλεονέκτημά της είναι ότι μπορεί να πιάσει και άγνωστες επιθέσεις. Το μειονέκτημά της είναι ότι χτυπά και για ασυνήθιστες αλλά νόμιμες συμπεριφορές, δημιουργώντας πολλούς εσφαλμένους συναγερμούς και την περίφημη κόπωση ειδοποιήσεων (alert fatigue), δηλαδή την κούραση των αναλυτών από τους πολλούς ψευδείς συναγερμούς (διάλεξη 1).

Η αντιπαράθεση των δύο προσεγγίσεων αποτυπώνεται σε μια αναλογία με δύο φρουρούς σε μια είσοδο. Ο πρώτος αναγνωρίζει μόνο συγκεκριμένα, καταγεγραμμένα πρόσωπα ως εισβολείς. Σταματά με βεβαιότητα όποιον βρίσκεται στη λίστα, αλλά αφήνει να περάσει ένας άγνωστος εισβολέας με μεταμφίεση. Ο δεύτερος κρίνει με βάση την ασυνήθιστη συμπεριφορά. Εντοπίζει τον άγνωστο, αλλά ενοχλεί και αθώους που απλώς φέρονται περίεργα. Κανένas από τους δύο δεν επαρκεί

μόνος του. Η λύση είναι η συνεργασία τους, με σαφείς κανόνες για τις περιπτώσεις που διαφωνούν. Αυτό ακριβώς είναι το υβριδικό σύστημα που κατασκευάστηκε στην παρούσα εργασία.

Ιδιαίτερη σημασία δόθηκε στην τιμότητα της αξιολόγησης, εξαιτίας μιας παγίδας που, αν δεν ληφθεί υπόψη, οδηγεί σε εντυπωσιακά αλλά πλαστά αποτελέσματα. Εάν ένα μοντέλο βαθμολογηθεί χρησιμοποιώντας ως σωστές απαντήσεις τα ίδια στοιχεία που του δόθηκαν για να αποφασίσει, τότε απλώς επαναλαμβάνει όσα ήδη γνωρίζει και παράγει τέλειο σκορ, το οποίο όμως στερείται αξίας. Η κατάσταση αντιστοιχεί στη χορήγηση σε έναν εξεταζόμενο των θεμάτων μαζί με τις λύσεις τους. Για την αποφυγή της, ο χαρακτηρισμός του κάθε χρονικού διαστήματος ως επίθεσης (η σωστή απάντηση) σχεδιάστηκε ώστε να προκύπτει από εντελώς ανεξάρτητη πηγή σε σχέση με τα στοιχεία που επεξεργάζεται το μοντέλο. Χάρη σε αυτό, οι μετρικές που αναφέρονται είναι ρεαλιστικές (για παράδειγμα, ο βαθμός F1 του υβριδικού συστήματος ανέρχεται σε 0.926) και αντικατοπτρίζουν πραγματική ικανότητα ανίχνευσης. Η στάση αυτή συνδέεται με τις αρχές της υπεύθυνης τεχνητής νοημοσύνης (διάλεξη 7). Τηρήσαμε τρεις:

- **Ερμηνευσιμότητα:** το σύστημα εξηγεί γιατί πήρε κάθε απόφαση. Δείχνει ποια χαρακτηριστικά της κίνησης μέτρησαν περισσότερο και δίνει έναν βαθμό κινδύνου σε κάθε στιγμή, αντί να λειτουργεί ως αδιαφανές μαύρο κουτί.
- **Λογοδοσία:** κάθε συναγεμμός συνοδεύεται από την πηγή του, δηλαδή από το ποιο σκέλος (ο κανόνας ή το μοντέλο) τον προκάλεσε, ώστε να μπορεί κανείς να τον ελέγξει.
- **Ειλικρινής αναφορά:** δεν κρύψαμε τις αστοχίες του συστήματος (όπως τους εσφαλμένους συναγεμμούς του μοντέλου) ούτε εξωραϊσαμε τα αποτελέσματα.

Ως πεδίο εφαρμογής επιλέξαμε έναν διακομιστή ιστού (web server) μέσα σε ένα εσωτερικό δίκτυο, ο οποίος δέχεται επιθέσεις από έναν εξωτερικό επιτιθέμενο. Το σενάριο αυτό είναι ρεαλιστικό και συνδυάζει σαφή νόμιμη λειτουργία (την εξυπηρέτηση ιστοσελίδων) με ένα ευρύ φάσμα δικτυακών επιθέσεων.

Η εργασία απαντά σε επτά ζητούμενα. Θα σχεδιάσουμε το εργαστήριο και θα περιγράψουμε τον αντίπαλο (Task 1), θα παραγάγουμε και θα χαρακτηρίσουμε την κίνηση (Task 2), θα φτιάξουμε κανόνες Snort (Task 3), θα εκπαιδεύσουμε μοντέλα μηχανικής μάθησης (Task 4), θα συνδυάσουμε τα δύο επίπεδα (Task 5), θα αξιολογήσουμε το σύστημα (Task 6) και θα μελετήσουμε πώς ένας επιτιθέμενος θα προσπαθούσε να το ξεγελάσει (Task 7). Υλοποιήσαμε επίσης δύο προαιρετικά tasks. Οι επόμενες ενότητες εξηγούν αναλυτικά καθένα από τα παραπάνω.

Task 1: Σχεδιασμός testbed και μοντέλο απειλής

Πριν ανιχνεύσει κανείς επιθέσεις, χρειάζεται ένα ασφαλές μέρος για να τις εκτελέσει και να τις μελετήσει. Αυτό ονομάζεται testbed, δηλαδή ένα ελεγχόμενο εργαστηριακό περιβάλλον στο οποίο γνωρίζουμε ακριβώς τι συμβαίνει κάθε στιγμή. Επειδή εμείς οι ίδιοι στήνουμε και τις επιθέσεις και τη νόμιμη κίνηση, ξέρουμε με βεβαιότητα ποια στιγμή ήταν επίθεση και ποια όχι, κάτι που είναι απαραίτητο για μια δίκαιη αξιολόγηση.

Τοπολογία και υπηρεσία-στόχος

Στήσαμε δύο ξεχωριστά δίκτυα αντί για ένα. Το εσωτερικό δίκτυο το ονομάζουμε HOME_NET και αντιστοιχεί στο εύρος διευθύνσεων 10.0.0.0/24 (δηλαδή τις διευθύνσεις που ξεκινούν από 10.0.0). Μέσα σε αυτό βρίσκεται ο διακομιστής-θύρα με διεύθυνση 10.0.0.10, ο οποίος εξυπηρετεί ιστοσελίδες στη θύρα 8080. Η θύρα είναι σαν τον αριθμό μιας πόρτας. Ένας υπολογιστής μπορεί να τρέχει πολλές υπηρεσίες ταυτόχρονα, και η θύρα ξεχωρίζει σε ποια υπηρεσία απευθύνεται κάθε αίτημα. Στο ίδιο εσωτερικό δίκτυο υπάρχει και ένας νόμιμος χρήστης (10.0.0.50). Το εξωτερικό δίκτυο, που το ονομάζουμε EXTERNAL_NET (δηλαδή οτιδήποτε δεν ανήκει στο HOME_NET), περιλαμβάνει τον επιτιθέμενο (192.168.56.20) και έναν νόμιμο εξωτερικό επισκέπτη (203.0.113.7).

Ο λόγος που επιλέξαμε δύο δίκτυα και όχι ένα είναι ουσιαστικός. Αν όλα τρέχανε στην ίδια μηχανή (σε localhost), τότε η έννοια, κίνηση που έρχεται από έξω, δεν θα είχε νόημα, αφού τα πάντα θα προέρχονταν από την ίδια διεύθυνση. Ένας από τους κανόνες μας ανιχνεύει ring που έρχεται από εκτός του εσωτερικού δικτύου. Με δύο δίκτυα, ο επιτιθέμενος βρίσκεται πράγματι απέξω, οπότε ο κανόνας έχει πραγματικό νόημα και μπορεί να ελεγχθεί.

Μοντέλο απειλής: Ποιον αντιμετωπίζουμε

Για να θωρακίσουμε ένα σύστημα, πρέπει πρώτα να περιγράψουμε τον αντίπαλο. Ο επιτιθέμενός μας μπορεί να στείλει οποιαδήποτε κίνηση θέλει προς τον διακομιστή από το εξωτερικό δίκτυο, δηλαδή πακέτα κάθε είδους και αιτήματα ιστού. Δεν έχει όμως προνομιακή πρόσβαση, δεν ελέγχει τον διακομιστή, ούτε τον αισθητήρα Snort, ούτε τα μοντέλα ή τα δεδομένα με τα οποία εκπαιδεύτηκαν. Στόχος του είναι η αναγνώριση του συστήματος (να δει τι τρέχει και πού), η υπερφόρτωσή του, και η επίθεση στην εφαρμογή ιστού. Ορίζουμε ως παραβίαση την παρουσία κακόβουλης δραστηριότητας μέσα σε ένα χρονικό διάστημα, ενώ ως σωστή λειτουργία ορίζουμε την ανίχνευση της επίθεσης χωρίς να μπλοκάρεται η νόμιμη κίνηση.

Πώς ξέρουμε τι ήταν επίθεση (ground truth)

Για να πούμε αν το σύστημα απάντησε σωστά, χρειαζόμαστε την αντικειμενική αλήθεια, δηλαδή ποιες στιγμές ήταν όντως επίθεση. Αυτή την αλήθεια την ονομάζουμε ground truth και την έχουμε από δύο ανεξάρτητες πηγές. Πρώτον, εμείς προγραμματίσαμε τότε τρέχει κάθε επίθεση και καταγράψαμε τις ακριβείς ώρες σε ένα αρχείο (logs/event_log.csv). Δεύτερον, γνωρίζουμε τη διεύθυνση του επιτιθέμενου. Χωρίζουμε τον χρόνο σε κομμάτια του ενός δευτερολέπτου, που τα λέμε παράθυρα (windows), και χαρακτηρίζουμε ένα παράθυρο ως επίθεση αν περιέχει έστω και ένα πακέτο του επιτιθέμενου, αλλιώς ως νόμιμο.

Γιατί κρατήσαμε χωριστά την αλήθεια από τα στοιχεία που βλέπει το μοντέλο

Εδώ βρίσκεται το πιο σημαντικό σημείο της μεθοδολογίας μας. Η αλήθεια (αν ένα παράθυρο ήταν επίθεση) καθορίζεται αποκλειστικά από τον χρόνο και τη διεύθυνση του επιτιθέμενου. Δεν καθορίζεται ποτέ από τα μετρήσιμα χαρακτηριστικά της κίνησης, όπως το πλήθος των πακέτων ή τα flags τους.

Ο λόγος είναι ο εξής. Εάν ο χαρακτηρισμός ενός παραθύρου ως επίθεσης προέκυπτε από έναν κανόνα του τύπου, αν τα πακέτα ξεπερνούν τον αριθμό X τότε πρόκειται για επίθεση, και στη συνέχεια το μοντέλο καλούνταν να ξεχωρίσει τις επιθέσεις κοιτώντας ακριβώς αυτόν τον αριθμό πακέτων, τότε θα ξαναμάθαινε απλώς τον ίδιο κανόνα. Θα παρήγαγε τέλειο σκορ, εντελώς πλαστό όμως, καθώς δεν θα είχε μάθει κάτι πραγματικό. Το πρόβλημα αυτό ονομάζεται διαρροή δεδομένων. Διατηρώντας τον χαρακτηρισμό ανεξάρτητο από τα χαρακτηριστικά, οι μετρικές είναι πραγματικές, εμφανίζονται αληθινά λάθη (τόσο χαμένες επιθέσεις όσο και εσφαλμένοι συναγερμοί), και ολόκληρη η ανάλυση που ακολουθεί αποκτά νόημα.

Πώς είναι φτιαγμένο το σύστημα

Το σύστημα δεν είναι ένα ενιαίο μεγάλο πρόγραμμα, αλλά μια σειρά από μικρά προγράμματα σε γλώσσα Python, το καθένα με μία δουλειά, σαν τα βήματα μιας γραμμής παραγωγής. Το **build_dataset.py** παράγει την κίνηση, το **feature_extraction.py** υπολογίζει τα χαρακτηριστικά και τα labels, το **snort_emulator.py** υλοποιεί το σκέλος των υπογραφών, το **train_model.py** εκπαιδεύει τα μοντέλα, το **hybrid_fusion.py** συνδυάζει τα δύο σήματα και το **evaluation.py** βγάζει τις μετρικές και τα γραφήματα. Ολόκληρη η αλυσίδα τρέχει με μία εντολή μέσω του **run_all.sh**.

Γιατί τα αποτελέσματα βγαίνουν ίδια κάθε φορά

Η παραγωγή της κίνησης και η εκπαίδευση των μοντέλων χρησιμοποιούν έναν σταθερό αριθμό τυχειότητας (τον αριθμό 1337). Ο αριθμός κάνει την τυχειότητα επαναλήψιμη, ώστε κάθε εκτέλεση να δίνει ακριβώς τα ίδια αποτελέσματα. Επιπλέον, όλη η ανάλυση γίνεται εκτός σύνδεσης (offline) πάνω σε ένα ενιαίο αρχείο καταγραφής κίνησης (pcap), δηλαδή σε μια ηχογράφηση της κίνησης που αναλύουμε ξανά και ξανά, αντί να στηριζόμαστε σε ζωντανή σύλληψη που θα άλλαζε κάθε φορά.

Αρχεία καταγραφής (logs/)

Ο φάκελος logs/ κρατά τα τεκμήρια του συστήματος. Ο Πίνακας 1 εξηγεί τι περιέχει και σε τι χρησιμεύει το κάθε αρχείο.

Αρχείο	Στήλες	Ρόλος
event_log.csv	attack_type, start_time, end_time, description	Πότε έτρεξε κάθε επίθεση. Πάνω σε αυτό στηρίζεται η επισήμανση των παραθύρων. Το διαβάζει το feature_extraction.py .
snort_alerts.csv	rel_time, sid, msg, src	Οι συναγερμοί του Snort, ένας ανά πακέτο. Αποδεικνύει ότι κάθε κανόνας πυροδοτείται.
snort_windows.csv	window_id, snort_detection, sids_fired	Το σήμα του Snort ανά παράθυρο. Είναι η γέφυρα προς το μοντέλο. Το hybrid_fusion.py τα ενώνει μέσω του window_id.

Αρχεία που υλοποιούν το Task 1: το αρχείο τεκμηρίωσης **TESTBED_DESIGN.md**, στο οποίο περιγράφεται η τοπολογία του εργαστηρίου και η μεθοδολογία και το πρόγραμμα **traffic_generation/build_dataset.py**, το οποίο όταν εκτελεστεί παράγει δύο αρχεία: το αρχείο κίνησης **pcaps/lab_traffic.pcap** (η ηχογράφηση όλης της κίνησης) και το χρονοδιάγραμμα των **επιθέσεων logs/event_log.csv** (το ground truth).

Task 2: Παραγωγή και επισήμανση κίνησης

Ένα σύστημα ανίχνευσης δεν μπορεί να αξιολογηθεί χωρίς δεδομένα. Παραγάγαμε λοιπόν μια ρεαλιστική ροή κίνησης διάρκειας περίπου 1.799 δευτερολέπτων (γύρω στη μισή ώρα), που περιέχει 24.110 πακέτα. Το πακέτο είναι η μικρότερη μονάδα δεδομένων που ταξιδεύει στο δίκτυο, σαν έναν φάκελο με λίγη πληροφορία μέσα. Από αυτά, τα 22.456 είναι πακέτα TCP (το πρωτόκολλο πάνω στο οποίο στηρίζεται ο ιστός) και τα 1.654 είναι ICMP (το πρωτόκολλο των ping).

Η νόμιμη κίνηση

Η νόμιμη κίνηση προσομοιώνει την καθημερινή ζωή ενός διακομιστή. Περιλαμβάνει κανονικές επισκέψεις σε ιστοσελίδες από τον εσωτερικό και τον εξωτερικό νόμιμο χρήστη, περιοδικά ping για έλεγχο ότι ο διακομιστής είναι ζωντανός, και κάθε τόσο κάποιες βαριές αλλά απολύτως νόμιμες ριπές, όπως όταν μια σελίδα φορτώνει πολλές εικόνες μαζί. Βάλαμε επίτηδες αυτές τις βαριές ριπές, ώστε το πρόβλημα να μην είναι τεχνητά εύκολο. Αν όλη η νόμιμη κίνηση ήταν ήσυχη και όλες οι επιθέσεις θορυβώδεις, ο διαχωρισμός θα ήταν τετριμμένος και τα αποτελέσματα παραπλανητικά.

Οι επιθέσεις

Υλοποιήσαμε έξι είδη επιθέσεων, καλύπτοντας το ζητούμενο των τεσσάρων κακόβουλων σεναρίων συν μία επίθεση χαμηλού και αργού ρυθμού, προσθέτοντας και μία κρυφή (stealth) παραλλαγή για να δείξουμε καθαρά τη συμπληρωματικότητα των ανιχνευτών. Για κάθε είδος εξηγούμε τι είναι και γιατί το κάνει ένας επιτιθέμενος:

- **SYN scan:** ο επιτιθέμενος στέλνει αιτήματα σύνδεσης σε πάρα πολλές θύρες, σαν να χτυπά μία μία όλες τις πόρτες για να δει ποιες είναι ανοιχτές. Είναι το πρώτο βήμα της αναγνώρισης.
- **ICMP flood:** πλημμυρίζει τον διακομιστή με ping μεγάλης συχνότητας, προσπαθώντας να τον υπερφορτώσει.
- **Connection flood:** ανοίγει πάρα πολλές πλήρεις συνδέσεις πολύ γρήγορα, εξαντλώντας τους πόρους του διακομιστή.
- **HTTP probe:** ζητά ύποπτες διευθύνσεις (π.χ. σελίδες διαχείρισης όπως /phpmyadmin/) και χρησιμοποιεί το αναγνωριστικό ενός γνωστού εργαλείου επιθέσεων (sqlmap), αφήνοντας χαρακτηριστικό αποτύπωμα.

- **Stealth web probe:** κάνει τα ίδια ύποπτα αιτήματα, αλλά αραιά και μεμονωμένα, ώστε ο όγκος να μοιάζει φυσιολογικός και να περνά απαρατήρητος.
- **Low-and-slow SQLi:** στέλνει μια επίθεση SQL injection κομμένη σε μικρά κομμάτια με καθυστερήσεις ανάμεσά τους, ώστε να μένει κάτω από τα όρια που ενεργοποιούν τους συναγερμούς. Είναι σχεδιασμένη ειδικά για να ξεφεύγει.

Από πακέτα σε παράθυρα και η ανισορροπία των κλάσεων

Δεν εξετάζουμε κάθε πακέτο χωριστά, αλλά ομαδοποιούμε τα πακέτα σε παράθυρα του ενός δευτερολέπτου και περιγράφουμε το καθένα με συνολικά μεγέθη (π.χ. πόσα πακέτα, πόσα ring κ.λπ.). Έτσι προκύπτουν 1.484 παράθυρα, από τα οποία τα 174 είναι επίθεση και τα 1.310 νόμιμα. Το ότι μόνο το 11,7% είναι επιθέσεις είναι σκόπιμο και ρεαλιστικό, γιατί στην πραγματικότητα οι επιθέσεις είναι σπάνιες. Αυτή η ανισορροπία έχει σημασία, γιατί μας υποχρεώνει να μην κοιτάμε μόνο τη γενική ακρίβεια. Αν κάποιος έλεγε πάντα δεν υπάρχει επίθεση, θα είχε δίκιο στο 88% των περιπτώσεων, χωρίς όμως να έχει πιάσει ούτε μία επίθεση. Γι' αυτό χρησιμοποιούμε πιο κατάλληλες μετρικές, όπως εξηγούμε στο Task 6 (διάλεξη 5).

Τύπος επίθεσης	Επεισόδια	Attack windows
SYN_SCAN	4	8
ICMP_FLOOD	3	12
CONN_FLOOD	3	8
HTTP_ATTACK	4	19
STEALTH_WEB	2	38
SLOW_ATTACK	2	89
Σύνολο	18	174

Αρχεία που υλοποιούν το Task 2: το πρόγραμμα **traffic_generation/build_dataset.py** παράγει την κίνηση, και το **ml_detector/feature_extraction.py** επεξεργάζεται αυτή την κίνηση και βγάζει το αρχείο **features.csv**, δηλαδή τον πίνακα με τα χαρακτηριστικά κάθε παραθύρου μαζί με την επισήμανσή του (επίθεση ή νόμιμο).

Task 3: Ανάπτυξη κανόνων Snort

Το Snort είναι ένα ευρέως διαδεδομένο, ανοιχτού κώδικα σύστημα ανίχνευσης εισβολών που λειτουργεί με υπογραφές. Διαβάζει την κίνηση του δικτύου και τη συγκρίνει με έναν κατάλογο κανόνων. Κάθε κανόνας ορίζει, στην ουσία, ότι εφόσον εντοπιστεί μια συγκεκριμένη μορφή κίνησης, καταγράφεται ο αντίστοιχος συναγερμός. Στην εργασία υλοποιήθηκαν πέντε κανόνες (στο αρχείο **rules/local.rules**), περισσότεροι από τους τρεις που ζητούνταν.

Ένας κανόνας έχει δύο βασικά μέρη. Το πρώτο είναι το τι αναζητά μέσα στο περιεχόμενο, για παράδειγμα μια συγκεκριμένη λέξη σε ένα αίτημα ιστού (αυτό ονομάζεται *content match*). Το δεύτερο είναι ένα κατώφλι συχνότητας (*detection_filter*), δηλαδή η πυροδότηση συναγερμού μόνο όταν κάτι συμβεί πολλές φορές σε σύντομο χρόνο. Το κατώφλι είναι σημαντικό, γιατί ένα μεμονωμένο *ping* είναι φυσιολογικό, ενώ εκατοντάδες *ping* στο δευτερόλεπτο συνιστούν επίθεση. Οι κανόνες που αναπτύχθηκαν είναι:

- **sid 1000001:** πιάνει αίτημα σε ύποπτη σελίδα διαχείρισης (*/phpmyadmin/*) με βάση το περιεχόμενο, χωρίς κατώφλι, ώστε να εντοπίζεται ακόμη και ένα μεμονωμένο τέτοιο αίτημα.
- **sid 1000002:** πιάνει το αποτύπωμα του εργαλείου *sqlmap* στην κεφαλίδα του αιτήματος.
- **sid 1000003:** πιάνει καταιγισμό από *ping* που έρχονται από έξω, με κατώφλι 30 στο δευτερόλεπτο ανά αποστολέα.
- **sid 1000004:** πιάνει τη σάρωση θυρών, με κατώφλι 40 αιτημάτων σύνδεσης σε 2 δευτερόλεπτα ανά αποστολέα.
- **sid 1000005:** πιάνει την πλημμύρα συνδέσεων, με κατώφλι 20 στο δευτερόλεπτο ανά αποστολέα.

Για να επιβεβαιώσουμε ότι δουλεύουν, τρέξαμε τους κανόνες πάνω στην ηχογραφημένη κίνηση και μετρήσαμε τους συναγερμούς. Και οι πέντε πυροδοτούνται, όπως δείχνει ο πίνακας παρακάτω. Πρέπει να διευκρινίσουμε μια πρακτική λεπτομέρεια. Για να μπορεί οποιοσδήποτε να αναπαραγάγει τα αποτελέσματα χωρίς να εγκαταστήσει το Snort, ενσωματώσαμε στο pipeline έναν πιστό προσομοιωτή των κανόνων (*snort_emulator.py*) που εφαρμόζει ακριβώς τα ίδια κατώφλια. Ο πραγματικός Snort τρέχει μία φορά, ξεχωριστά, μόνο για να τραβήξουμε τα στιγμιότυπα των συναγερμών (οδηγίες στο *SNORT_GUIDE.md*), και τα αποτελέσματά του συμφωνούν με του προσομοιωτή.

sid	Κανόνας	Alerts
1000001	WEB Suspicious URI (phpmyadmin)	46
1000002	WEB sqlmap User-Agent	95
1000003	ICMP Echo flood (external)	1415
1000004	TCP SYN scan / port sweep	111
1000005	TCP connection flood (burst)	171

Επαλήθευση με πραγματικό Snort

Πέρα από τον προσομοιωτή, οι κανόνες επαληθεύτηκαν και με πραγματικό Snort 3.12.2 σε περιβάλλον Kali Linux, εκτελώντας τον offline πάνω στο ίδιο pcap. Αναλύθηκαν και τα 24.110 πακέτα και πυροδοτήθηκαν συνολικά 1607 alerts. Οι τρεις κανόνες όγκου ενεργοποιήθηκαν κανονικά (ICMP flood 1412, SYN scan 73, connection flood 122), ενώ οι δύο κανόνες περιεχομένου (phpmyadmin, sqlmap) δεν ενεργοποιήθηκαν στη συγκεκριμένη έκδοση, καθώς ο μηχανισμός http_inspect του Snort 3.12 δεν αντιστοιχίζει το συνθετικό HTTP payload με τον τρόπο που το κάνει ο προσομοιωτής. Τα σχετικά στιγμιότυπα παρατίθενται στον φάκελο screenshots. Οι μετρικές της μηχανικής μάθησης και του υβριδικού συστήματος δεν επηρεάζονται, καθώς προκύπτουν από το πλήρες σύνολο κανόνων μέσω του προσομοιωτή.

Εκτέλεση του πραγματικού Snort 3.12.2 σε Kali Linux πάνω στο pcap: φόρτωση των πέντε κανόνων και πλήρης σύνοψη ανάλυσης (24.110 πακέτα, 1607 alerts):

```

[~(kali@kali)-[~/Downloads/MTE25612_Snort]
$ snort -c snort_kali.lua -R rules_kali.rules -r pcaps/lab_traffic.pcap -A alert_fast -l logs -k none
o")~  Snort++ 3.12.2.0

Loading snort_kali.lua:
  http_inspect
  alert_fast
  output
  ips
  stream_tcp
  search_engine
  packets
  active
  alerts
  daq
  decode
  host_cache
  host_tracker
  hosts
  process
  network
  stream
  stream_icmp
  binder
Finished snort_kali.lua:
Loading rule args:
Loading rules_kali.rules:
Finished rules_kali.rules:
Finished rule args:

ips policies rule stats
  id loaded shared enabled file
  1 5 0 5 snort_kali.lua

rule counts
  total rules loaded: 5
  text rules: 5
  option chains: 5
  chain headers: 2

port rule counts
  tcp udp icmp ip
  any 0 0 1 0
  dst 2 0 0 0
  total 2 0 1 0

service rule counts
  http: 2 0
  http2: 2 0
  http3: 2 0
  total: 6 0

fast pattern groups
  to_server: 6

search engine (ac_bnfa)
  instances: 6
  patterns: 6
  pattern chars: 54
  num states: 54
  num match states: 6
  memory scale: KB
  total memory: 8.22656
  pattern memory: 0.28125
  match list memory: 0.689375
  transition memory: 6.58594
  fast pattern only: 6

pcap DAQ configured to read-file.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] pcaps/lab_traffic.pcap
-- [0] pcaps/lab_traffic.pcap

Packet Statistics
daq
  pcaps: 1
  received: 24110
  analyzed: 24110
  allow: 24110
  rx_bytes: 4214664

codec
  total: 24110 (100.000%)
  icmp4: 1654 ( 6.860%)
  ipv4: 24110 (100.000%)
  tcp: 22456 ( 93.140%)

Module Statistics
binder
  raw_packets: 4147
  new_flows: 2358
  no_match: 1815
  inspects: 6505

detection
  analyzed: 24110
  hard_evals: 10998
  alerts: 1607
  total_alerts: 1607
  logged: 1607

ips_actions
  alert: 1607

normalizer
  test_tcp_trim_win: 8668
  test_tcp_ips_data: 69

search_engine
  non_qualified_events: 9464
  qualified_events: 1534

stream
  flows: 2358
  total_prunes: 21
  idle_prunes_proto_timeout: 3
  flow_closed: 18
  icmp_timeout_prunes: 3
  no_flow_tcp_rst: 811
  no_flow_unwanted: 3336

stream_icmp
  sessions: 4
  max: 4
  created: 4
  released: 4

stream_tcp
  sessions: 2354
  max: 2354
  created: 2354
  released: 2354
  timeouts: 18
  instantiated: 2354
  setups: 2354
  syn_ack_trackers: 2354
  segs_queued: 2400
  segs_released: 2400
  overlaps: 115
  payload_fully_trimmed: 46
  syn_acks: 2354
  fins: 4590
  max_segs: 3
  max_bytes: 200
  zero_win_probes: 6547

Summary Statistics
timing
  runtime: 00:00:00
  seconds: 0.180730
  pkts/sec: 133403
  Mbits/sec: 187
o")~  Snort exiting
```

Δείγμα των συναγερμών (logs/alert_fast.txt) και πλήθος alerts ανά κανόνα:

```
(kali@kali)~/Downloads/MTE25012_Snort
$ cat logs/alert_fast.txt | head -30
for s in 1000001 1000002 1000003 1000004 1000005; do echo -n "$s: "; grep -c $s logs/alert_fast.txt; done
05/23-17:38:23.469039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.475039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.481039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.487039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.493039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.499039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.505039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.511039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.517039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.523039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.529039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.535039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.541039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.547039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.553039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.559039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.565039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.571039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.577039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.583039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.589039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.595039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.601039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.607039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.613039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.619039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.625039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.631039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.637039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
05/23-17:38:23.643039 [**] [1:1000003:1] "ICMP Echo flood from external host" [**] [Priority: 0] {ICMP} 192.168.56.20 → 10.0.0.10
1000001: 0
1000002: 0
1000003: 1412
1000004: 73
1000005: 122
(kali@kali)~/Downloads/MTE25012_Snort
```

Πλήθος alerts ανά κανόνα (sid): ICMP flood 1412, SYN scan 73, connection flood 122:

```
(kali@kali)~/Downloads/MTE25012_Snort
$ for s in 1000001 1000002 1000003 1000004 1000005; do echo -n "$s: "; grep -c $s logs/alert_fast.txt; done
1000001: 0
1000002: 0
1000003: 1412
1000004: 73
1000005: 122
(kali@kali)~/Downloads/MTE25012_Snort
$
```

Αρχεία που υλοποιούν το Task 3: το αρχείο `rules/local.rules` περιέχει τους πέντε κανόνες Snort και το `rules/snort.lua` τη ρύθμιση για την εκτέλεσή τους, ενώ το πρόγραμμα `ml_detector/snort_emulator.py` εφαρμόζει τους κανόνες στην κίνηση και παράγει δύο αρχεία: το `logs/snort_alerts.csv` με τους συναγερμούς ανά πακέτο και το `logs/snort_windows.csv` με το σήμα του Snort ανά παράθυρο.

Task 4: Features και μοντέλα μηχανικής μάθησης

Τι είναι τα features

Ένα μοντέλο μηχανικής μάθησης δεν βλέπει ωμά πακέτα, βλέπει αριθμούς. Feature είναι ένας αριθμός που περιγράφει κάτι για ένα παράθυρο του ενός δευτερολέπτου. Υπολογίσαμε 17 τέτοια features, διαλεγμένα ώστε να σχετίζονται με τις επιθέσεις. Χονδρικά χωρίζονται σε ομάδες: μεγέθη όγκου (πόσα πακέτα και πόσα bytes, μέσο μέγεθος πακέτου), μετρήσεις των flags TCP (μικρές ενδείξεις που δείχνουν αν ένα πακέτο ανοίγει, κλείνει ή μεταφέρει δεδομένα μια σύνδεση), πλήθος ring, πόσοι διαφορετικοί αποστολείς και πόσες διαφορετικές θύρες εμφανίστηκαν, και χρονικά

χαρακτηριστικά (πόσο απέχουν χρονικά τα πακέτα μεταξύ τους). Τα χρονικά χαρακτηριστικά είναι ιδιαίτερα χρήσιμα για τις αργές επιθέσεις, οι οποίες δεν ξεχωρίζουν από τον όγκο τους αλλά από τον ρυθμό τους.

Πώς εκπαιδεύεται και πώς εξετάζεται ένα μοντέλο

Χωρίσαμε τα δεδομένα σε δύο μέρη, 70% για εκπαίδευση και 30% για εξέταση. Το μοντέλο μαθαίνει μόνο από το πρώτο μέρος και εξετάζεται στο δεύτερο, το οποίο δεν έχει ξαναδεί, όπως ακριβώς ένας μαθητής διαβάζει σε κάποια θέματα και εξετάζεται σε καινούργια. Φροντίσαμε ο διαχωρισμός να διατηρεί την ίδια αναλογία επιθέσεων και στα δύο μέρη. Μερικά μοντέλα χρειάζονται τα χαρακτηριστικά σε κοινή κλίμακα, γι' αυτό τα κανονικοποιούμε, και μάθαμε την κανονικοποίηση αυτή αποκλειστικά από το μέρος της εκπαίδευσης, ώστε να μην τρυπώσει καμία πληροφορία από την εξέταση (διάλεξη 4). Όλες οι μετρικές που αναφέρουμε προέρχονται από το μέρος της εξέτασης (446 παράθυρα, εκ των οποίων 52 επιθέσεις).

Τα τρία μοντέλα

Δοκιμάσαμε τρία μοντέλα, όλα από την ύλη του μαθήματος (διαλέξεις 3 και 4). Το Random Forest είναι ένα σύνολο από πολλά μικρά δέντρα αποφάσεων που ψηφίζουν και θεωρείται το πιο αξιόπιστο σημείο εκκίνησης για δεδομένα σε μορφή πίνακα (διάλεξη 4). Το Logistic Regression είναι ένας απλός σταθμισμένος τύπος που είναι εύκολος στην ερμηνεία. Το Isolation Forest είναι διαφορετικής φιλοσοφίας, δεν χρειάζεται καθόλου παραδείγματα επιθέσεων, μαθαίνει μόνο πώς μοιάζει το φυσιολογικό και σημαίνει ό,τι ξεχωρίζει ως ανωμαλία.

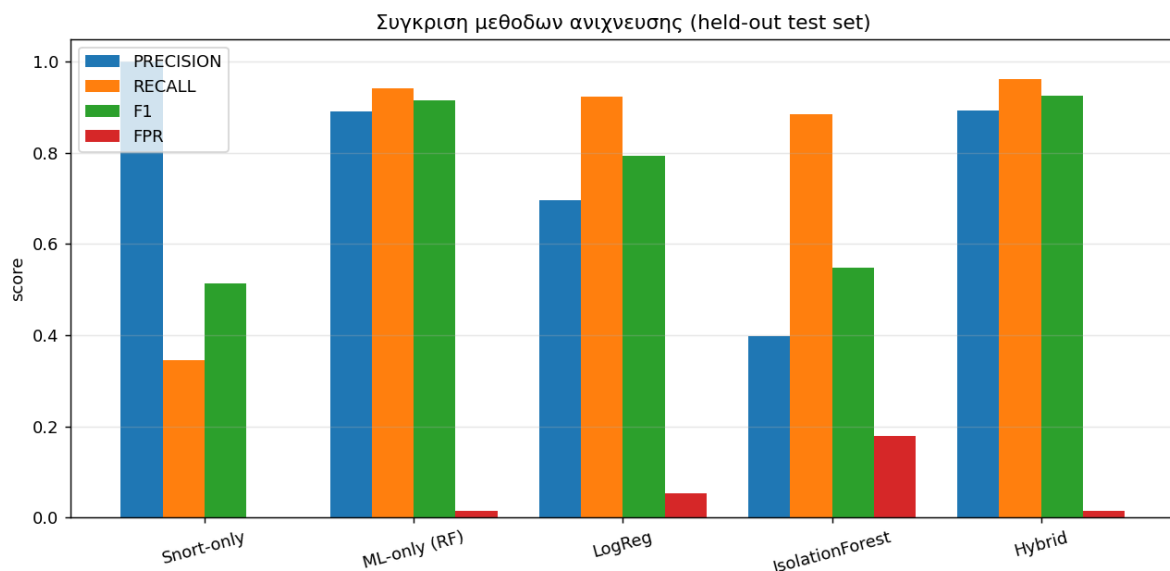
Πριν διαβάσουμε τα αποτελέσματα, χρειάζονται τέσσερις έννοιες. Η ακρίβεια (precision) απαντά, από όσα σήμανε το σύστημα ως επίθεση, πόσα ήταν όντως επίθεση. Η ανάκληση (recall) απαντά, από όσες επιθέσεις όντως έγιναν, πόσες έπιασε το σύστημα. Το F1 είναι μια ισορροπία των δύο σε έναν αριθμό. Το ποσοστό εσφαλμένων συναγερμών (FPR) δείχνει πόσο συχνά χτυπά καμπανάκι σε νόμιμη κίνηση. Ιδανικά επιδιώκεται υψηλή ακρίβεια και υψηλή ανάκληση ταυτόχρονα.

Μοντέλο	Precision	Recall	F1	FPR	AUC
Random Forest	0.891	0.942	0.916	0.015	0.991
Logistic Regression	0.696	0.923	0.793	0.053	0.968

Μοντέλο	Precision	Recall	F1	FPR	AUC
Isolation Forest	0.397	0.885	0.548	0.178	0.908

Το Random Forest δίνει την καλύτερη ισορροπία ($F1=0.916$). Το Logistic Regression πιάνει σχεδόν όλες τις επιθέσεις, αλλά χτυπά και πιο πολλές φορές άδικα ($FPR=0.053$), κάτι αναμενόμενο για ένα απλό γραμμικό μοντέλο όταν τα δεδομένα δεν χωρίζονται με ευθεία γραμμή. Το Isolation Forest, επειδή δεν είδε ποτέ παραδείγματα επιθέσεων, έχει καλή ανάκληση αλλά χαμηλή ακρίβεια (0.397), δηλαδή σημαίνει πολλά φυσιολογικά αλλά ασυνήθιστα παράθυρα ως ανωμαλίες. Αυτό επιβεβαιώνει μια βασική διαπίστωση, ότι το ασυνήθιστο δεν είναι απαραίτητως και κακόβουλο (διάλεξη 3). Αξίζει να τονιστεί ότι καμία μετρική δεν είναι τέλεια, γεγονός που αποτελεί θετική ένδειξη, καθώς επιβεβαιώνει την αποφυγή της διαρροής δεδομένων και τη μέτρηση πραγματικής ικανότητας.

Εξετάστηκε επίσης ποια χαρακτηριστικά μέτρησαν περισσότερο στις αποφάσεις του Random Forest. Στην κορυφή βρέθηκαν τα χρονικά χαρακτηριστικά (πόσο απέχουν τα πακέτα μεταξύ τους), ο αριθμός των διαφορετικών αποστολών και το πλήθος των πακέτων, κάτι που ταιριάζει απόλυτα με τη φύση των επιθέσεων. Το παρακάτω σχήμα συγκρίνει οπτικά όλες τις μεθόδους:



Αρχεία που υλοποιούν το Task 4: το πρόγραμμα `ml_detector/feature_extraction.py` υπολογίζει τα χαρακτηριστικά κάθε παραθύρου, και το `ml_detector/train_model.py` εκπαιδεύει τα μοντέλα. Η εκτέλεσή του παράγει το `predictions.csv` με τις προβλέψεις, το `ml_metrics.json` με τις μετρικές, καθώς και τα εκπαιδευμένα μοντέλα σε αρχεία (`rf_model.pkl` το Random Forest, `scaler.pkl` τον κανονικοποιητή και `iforest.pkl` το Isolation Forest).

Task 5: Υβριδική σύντηξη (Hybrid Fusion)

Σύντηξη (fusion) σημαίνει να συνδυάσουμε τις γνώμες των επιμέρους ανιχνευτών σε μία τελική απόφαση. Δώσαμε σε κάθε παράθυρο έναν βαθμό κινδύνου, συνδυάζοντας τρία σήματα με βάρη: $\text{κίνδυνος} = 0.55 \times (\text{σκορ Random Forest}) + 0.30 \times (\text{σήμα Snort}) + 0.15 \times (\text{σκορ Isolation Forest})$

Τα βάρη δεν είναι τυχαία. Δώσαμε το μεγαλύτερο βάρος στο Random Forest επειδή ήταν το πιο αξιόπιστο μοντέλο, μεσαίο βάρος στο Snort επειδή είναι πολύ ακριβές όταν χτυπά και το μικρότερο βάρος στο Isolation Forest επειδή είναι το πιο θορυβώδες. Τον βαθμό κινδύνου τον μεταφράζουμε σε τρεις στάθμες, σαν φανάρι: χαμηλή (LOW), μεσαία (MEDIUM) και υψηλή (HIGH).

Η τελική απόφαση, αν δηλαδή θα σημάνει συναγερμός, ορίζεται ως η ένωση των δύο πιο ακριβών ανιχνευτών, δηλαδή χτυπάμε αν χτυπήσει ο Snort ή αν το Random Forest πει επίθεση. Έτσι ο καθένας καλύπτει το τυφλό σημείο του άλλου. Το Isolation Forest δεν χτυπά μόνο του, γιατί θα ρίχναμε πολλούς εσφαλμένους συναγερμούς. Χρησιμοποιεί μόνο για να ανεβάσει τη στάθμη σοβαρότητας, ως ένα είδος επιτήρησης.

Τι γίνεται όταν οι δύο ανιχνευτές διαφωνούν

Ορίσαμε ρητούς κανόνες για τις διαφωνίες, γιατί εκεί κρίνεται η αξία ενός υβριδικού συστήματος:

- **C1, όταν μιλά η υπογραφή:** αν χτυπήσει ο Snort αλλά το μοντέλο έχει χαμηλό σκορ, εμπιστευόμαστε τον κανόνα (η υπογραφή είναι σίγουρη ένδειξη) και ανεβάζουμε τη στάθμη σε τουλάχιστον HIGH.
- **C2, όταν μιλά μόνο η ανωμαλία:** αν δεν χτυπήσει ούτε ο Snort ούτε το Random Forest, αλλά το Isolation Forest δείχνει έντονη ανωμαλία, βάζουμε το παράθυρο σε στάθμη MEDIUM για παρακολούθηση, με χαμηλότερη βεβαιότητα.
- **C3, όταν μιλά μόνο το μοντέλο:** αν χτυπήσει το Random Forest ενώ ο Snort σιωπά, εμπιστευόμαστε το μοντέλο. Εδώ ακριβώς πιάνεται η αργή επίθεση, στην οποία ο Snort είναι τυφλός επειδή το περιεχόμενο έρχεται κομματιασμένο.

Αρχεία που υλοποιούν το Task 5: το πρόγραμμα `ml_detector/hybrid_fusion.py` συνδυάζει το σήμα του Snort με τα σκορ των μοντέλων και παράγει το `hybrid_results.csv`, που περιέχει τον

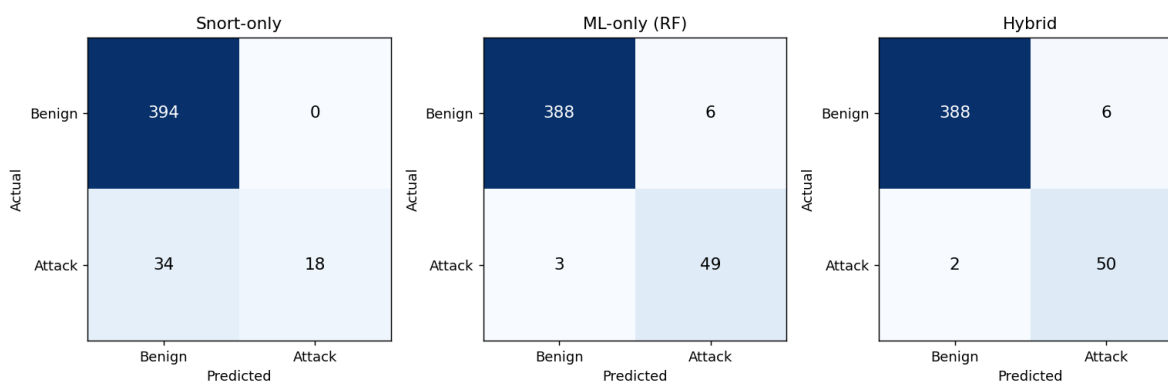
βαθμό κινδύνου και την τελική απόφαση για κάθε παράθυρο, καθώς και το **conflicts.json**, που καταγράφει τις περιπτώσεις διαφωνίας των δύο ανιχνευτών.

Task 6: Αξιολόγηση

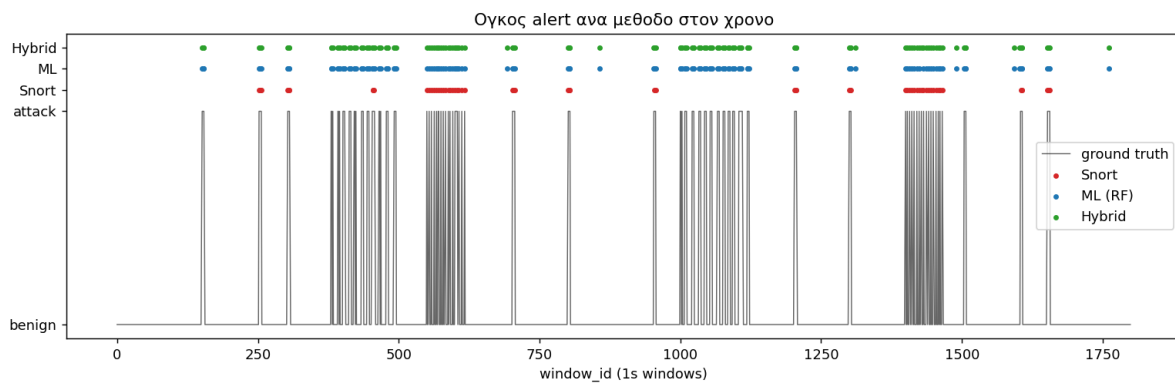
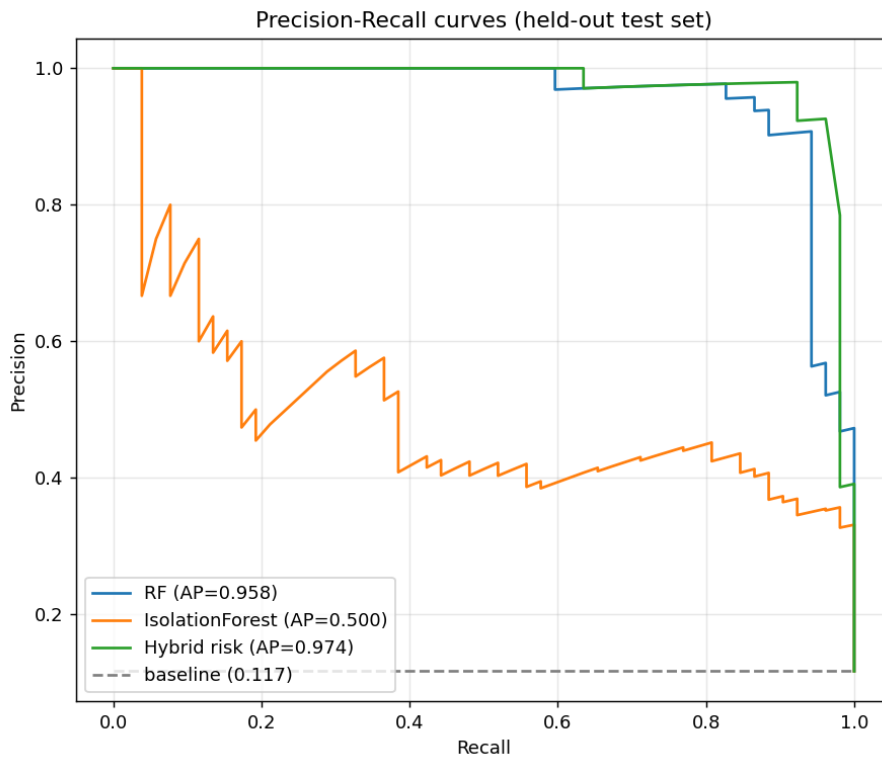
Ακολουθεί η σύγκριση των τριών προσεγγίσεων ως προς την επίδοσή τους. Ο Πίνακας 5 συγκρίνει τις τρεις στο σύνολο εξέτασης. Ο Snort μόνος του έχει τέλεια ακρίβεια (1.000), δηλαδή δεν χτυπά ποτέ άδικα, αλλά χαμηλή ανάκληση (0.346), δηλαδή χάνει τις περισσότερες επιθέσεις, ιδίως τις αργές και τις κρυφές. Το μοντέλο μόνο του (Random Forest) ανεβάζει κατακόρυφα την ανάκληση. Το υβριδικό πετυχαίνει το καλύτερο F1 (0.926), πιάνοντας μία επιπλέον επίθεση σε σχέση με το Random Forest, χωρίς να χάσει σε ακρίβεια.

Μέθοδος	TP	FP	FN	Precision	Recall	F1	FPR
Snort-only	18	0	34	1.000	0.346	0.514	0.000
ML-only (RF)	49	6	3	0.891	0.942	0.916	0.015
Hybrid	50	6	2	0.893	0.962	0.926	0.015

Οι στήλες TP, FP, FN αξίζει να εξηγηθούν, γιατί είναι ο πυρήνας κάθε αξιολόγησης (διάλεξη 5). TP (true positive) είναι μια επίθεση που πιάστηκε σωστά. FN (false negative) είναι μια επίθεση που διέφυγε. FP (false positive) είναι ένας εσφαλμένος συναγερμός σε νόμιμη κίνηση. Το ιδανικό σύστημα έχει πολλά TP και ελάχιστα FN και FP. Το παρακάτω σχήμα δείχνει τους λεγόμενους πίνακες σύγχυσης, που είναι απλώς αυτή η καταμέτρηση σε μορφή πίνακα για κάθε μέθοδο.



Επειδή οι επιθέσεις είναι σπάνιες, δώσαμε έμφαση στις καμπύλες ακρίβειας-ανάκλησης (Precision-Recall) αντί για τις πιο συνηθισμένες καμπύλες ROC, καθώς οι πρώτες είναι πιο κατατοπιστικές όταν η μία κλάση είναι σπάνια (διάλεξη 5). Το πρώτο σχήμα δείχνει ότι ο υβριδικός βαθμός κινδύνου υπερτερεί (μέση ακρίβεια $AP=0.974$). Το δεύτερο σχήμα δείχνει πότε χτυπά η κάθε μέθοδος στη διάρκεια του χρόνου. Προκύπτει ότι ο Snort χτυπά σπάνια αλλά σωστά, ενώ το υβριδικό καλύπτει και τις μεγάλες περιόδους της αργής επίθεσης που ο Snort αγνοεί.



Τρεις περιπτώσεις όπου οι ανιχνευτές διαφωνούν

Οι παρακάτω περιπτώσεις προέρχονται από πραγματικά παράθυρα και δείχνουν ξεκάθαρα γιατί κανένας ανιχνευτής δεν αρκεί μόνος.

- **Το μοντέλο πιάνει, ο Snort χάνει:** σε 98 παράθυρα το Random Forest πιάνει επιθέσεις που ο Snort χάνει. Πρόκειται κυρίως για την αργή επίθεση (κομματιασμένο περιεχόμενο και ρυθμός κάτω από τα κατώφλια) αλλά και για την αρχή της σάρωσης θυρών. Χαρακτηριστικό είναι το παράθυρο 151, με 122 διαφορετικές θύρες. Το μοντέλο το πιάνει αμέσως, ενώ ο Snort δεν έχει προλάβει να μαζέψει αρκετά αιτήματα για να ξεπεράσει το κατώφλι του.
- **Ο Snort πιάνει, το μοντέλο χάνει:** ο Snort πιάνει κρυφά αιτήματα που το μοντέλο χάνει. Στο παράθυρο 1428 το σκορ του μοντέλου είναι μόλις 0.12, γιατί ο όγκος μοιάζει με βαριά αλλά νόμιμη περιήγηση. Ο Snort όμως αναγνωρίζει το αποτύπωμα του sqlmap μέσα στο περιεχόμενο και χτυπά. Χάρη στον κανόνα C1, το υβριδικό το κατατάσσει σε HIGH.
- **Εσφαλμένοι συναγερμοί του μοντέλου:** σε 7 παράθυρα το Random Forest χτυπά άδικα ενώ ο Snort μένει καθαρός (π.χ. στα παράθυρα 692 και 857, με βαριά αλλά νόμιμη κίνηση). Η τέλεια ακρίβεια του Snort εξισορροπεί την τάση του μοντέλου για εσφαλμένους συναγερμούς.

Πόσο να εμπιστευόμαστε τα νούμερα

Οι μετρικές πρέπει να διαβάζονται με επίγνωση του μεγέθους του δείγματος. Με 52 επιθέσεις στο σύνολο εξέτασης, κάθε λάθος ταξινόμηση μετακινεί την ανάκληση κατά περίπου δύο μονάδες. Επομένως, οι μικρές διαφορές μεταξύ Random Forest και υβριδικού δεν πρέπει να υπερτονίζονται. Η ουσιαστική διαφορά φαίνεται καθαρότερα στην ανάλυση των διαφωνιών σε όλη τη χρονοσειρά, όπου το μοντέλο πιάνει μόνο του 98 παράθυρα και ο Snort ένα. Το ότι καμία μετρική δεν φτάνει στο τέλειο 1.000 είναι δείγμα υγιούς μεθοδολογίας, όχι αδυναμίας (διάλεξη 4).

Αρχεία που υλοποιούν το Task 6: το πρόγραμμα `ml_detector/evaluation.py` υπολογίζει τις μετρικές και το `evaluation.json` με τα αναλυτικά αποτελέσματα, ενώ το αρχείο `CASE_STUDIES.md` τεκμηριώνει τις τρεις μελέτες περίπτωσης.

Task 7: Τεχνικές απόκρυψης και κριτική αποτίμηση

Ένας έξυπνος επιτιθέμενος δεν επιτίθεται στα τυφλά, προσπαθεί να μη γίνει αντιληπτός. Εξετάσαμε τρεις τεχνικές απόκρυψης και ποιον ανιχνευτή ξεγελά η καθεμία:

- **Κομμάτιασμα του περιεχομένου (αργή επίθεση):** η υπογραφή σπάει σε πολλά πακέτα, ώστε κανένα να μην την περιέχει ολόκληρη. Ξεγελά κυρίως τον Snort, που ψάχνει το περιεχόμενο, όχι το μοντέλο, που κοιτά στατιστικά.
- **Κρύψιμο μέσα στον χαμηλό όγκο:** αραιά, μεμονωμένα αιτήματα ώστε ο όγκος να φαίνεται κανονικός. Ξεγελά κυρίως το μοντέλο, που βασίζεται σε συνολικά μεγέθη, ενώ ο Snort εξακολουθεί να πιάνει το περιεχόμενο.
- **Παραμονή κάτω από τα κατώφλια:** διατήρηση του ρυθμού κάτω από τα όρια των κανόνων. Ξεγελά τους κανόνες που στηρίζονται σε κατώφλια, ενώ το μοντέλο μπορεί να πιάσει το μοτίβο από άλλα χαρακτηριστικά, όπως ο αριθμός των θυρών.

Το βασικό συμπέρασμα είναι ότι κάθε κόλπο που εξουδετερώνει τον έναν ανιχνευτή αφήνει ίχνη ορατά στον άλλον. Αυτός ακριβώς είναι ο λόγος που φτιάξαμε υβριδικό σύστημα. Η ένωση των δύο σημάτων ανεβάζει την ανάκληση, ενώ ο χειρισμός των διαφωνιών κρατά χαμηλά τους εσφαλμένους συναγερμούς. Η μη αυτόματη κλιμάκωση των ενδείξεων που προέρχονται μόνο από ανωμαλία (κανόνας C2), μαζί με την τέλεια ακρίβεια των υπογραφών, περιορίζουν την κόπωση ειδοποιήσεων.

Οφείλουμε όμως να είμαστε ειλικρινείς για τα όρια. Το μοντέλο εκπαιδεύτηκε σε γνωστούς τύπους επιθέσεων. Απέναντι σε μια πραγματικά άγνωστη επίθεση (zero-day), η επιβλεπόμενη μάθηση δεν εγγυάται ανίχνευση, ενώ η μη επιβλεπόμενη προσφέρει κάποια κάλυψη με κόστος τη χαμηλή ακρίβεια. Ένας επιτιθέμενος που γνωρίζει πώς δουλεύει το σύστημα θα μπορούσε να φτιάξει κίνηση που να μιμείται στατιστικά τη νόμιμη, μια τεχνική που ονομάζεται ανταγωνιστική απόκρυψη (adversarial evasion, διάλεξη 2), απέναντι στην οποία κανένα από τα δύο σκέλη δεν είναι από μόνο του ανθεκτικό.

Προαιρετικό Task 1: Επιβλεπόμενη έναντι μη επιβλεπόμενης μάθησης

Η επέκταση αυτή συγκρίνει δύο θεμελιωδώς διαφορετικές προσεγγίσεις μηχανικής μάθησης, ώστε να αναδειχθεί μια βασική αντιστάθμιση της ασφάλειας. Στην επιβλεπόμενη μάθηση (Random Forest, Logistic Regression) το μοντέλο εκπαιδεύεται με χαρακτηρισμένα παραδείγματα, γνωρίζει δηλαδή εκ των προτέρων ποια κίνηση είναι επίθεση και ποια όχι. Στη μη επιβλεπόμενη μάθηση (Isolation Forest) το μοντέλο δεν βλέπει καμία ετικέτα, μαθαίνει μόνο πώς μοιάζει η φυσιολογική κίνηση και σημαίνει ό,τι αποκλίνει από αυτήν.

Η διαφορά αποτυπώνεται στις μετρικές. Τα δύο επιβλεπόμενα μοντέλα επιτυγχάνουν σαφώς υψηλότερη ακρίβεια (0.891 το Random Forest, 0.696 το Logistic Regression), επειδή, έχοντας δει

παραδείγματα επιθέσεων, χαράζουν πιο ακριβή όρια ανάμεσα στη νόμιμη και την κακόβουλη κίνηση. Το Isolation Forest, αντιθέτως, εμφανίζει χαμηλή ακρίβεια (0.397) και υψηλό ποσοστό εσφαλμένων συναγερμών ($FPR=0.178$), γιατί χαρακτηρίζει ως ανωμαλία κάθε ασυνήθιστο παράθυρο, ακόμη και όταν πρόκειται για νόμιμη αλλά σπάνια συμπεριφορά. Επιβεβαιώνεται έτσι έμπρακτα η αρχή ότι το ασυνήθιστο δεν ταυτίζεται με το κακόβουλο (διάλεξη 3).

Το εύρημα έχει πρακτική σημασία. Στα πραγματικά περιβάλλοντα ασφάλειας οι επιβεβαιωμένες ετικέτες επιθέσεων είναι σπάνιες και δαπανηρές, γεγονός που καθιστά ελκυστική τη μη επιβλεπόμενη μάθηση, καθώς δεν τις απαιτεί. Η χαμηλή της ακρίβεια όμως την καθιστά ακατάλληλη ως αυτόνομο μηχανισμό συναγερμού και καταλληλότερη ως βοηθητικό επίπεδο επιτήρησης και προτεραιοποίησης. Αυτόν τον ρόλο αναλαμβάνει στο επίπεδο σύντηξης μέσω του κανόνα C2, όπου το Isolation Forest δεν παράγει αυτόνομο συναγερμό αλλά μόνο ανεβάζει τη στάθμη σοβαρότητας. Αντιστάθμιση παρατηρείται και μεταξύ των δύο επιβλεπόμενων μοντέλων, καθώς το Random Forest, ως μη γραμμικό, πετυχαίνει καλύτερη ακρίβεια, ενώ το Logistic Regression είναι απλούστερο και πιο ερμηνεύσιμο, με τίμημα περισσότερους εσφαλμένους συναγερμούς.

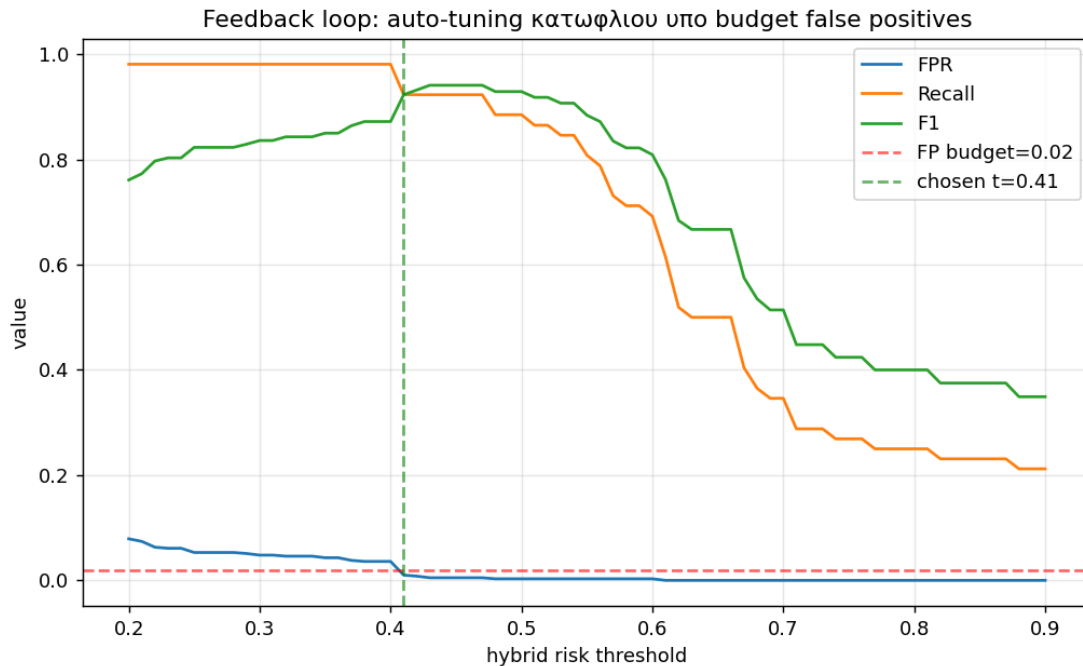
Αρχεία που υλοποιούν το προαιρετικό Task 1: το `ml_detector/train_model.py` εκπαιδεύει και τα τρία μοντέλα, ενώ το **`ml_detector/evaluation.py`** παράγει τη συγκριτική τους αποτίμηση, που αποτυπώνεται στα σχήματα 1 και 3.

Προαιρετικό Task 2: Βρόχος ανατροφοδότησης (feedback loop)

Το κατώφλι πάνω από το οποίο το σύστημα σημαίνει συναγερμό δεν αποτελεί σταθερή παράμετρο, αλλά επιλογή που εξαρτάται από το σχετικό κόστος ενός εσφαλμένου συναγερμού έναντι μιας χαμένης επίθεσης (διάλεξη 5). Για τον λόγο αυτό υλοποιήθηκε ένας μηχανισμός ανατροφοδότησης, ο οποίος ρυθμίζει αυτόματα το κατώφλι απόφασης του υβριδικού βαθμού κινδύνου, ώστε το ποσοστό εσφαλμένων συναγερμών να παραμένει κάτω από ένα προκαθορισμένο λειτουργικό όριο, εδώ ορισμένο στο 2%.

Ο μηχανισμός σαρώνει διαδοχικές τιμές κατωφλίου, μετρά για καθεμία το ποσοστό εσφαλμένων συναγερμών, την ανάκληση και τον βαθμό F1, και επιλέγει τη χαμηλότερη τιμή που ικανοποιεί τον περιορισμό. Στα δεδομένα καταλήγει στην τιμή 0.41, η οποία αποδίδει ποσοστό εσφαλμένων συναγερμών 0.010 με $F1=0.923$. Πρόκειται για μια απλή μορφή προσαρμοστικού ελέγχου, η οποία θα μπορούσε να επεκταθεί ώστε να ενημερώνει το κατώφλι με βάση την

ανατροφοδότηση των αναλυτών με την πάροδο του χρόνου. Το ακόλουθο σχήμα αποτυπώνει τη σάρωση και το σημείο λειτουργίας που επιλέχθηκε:



Αρχεία που υλοποιούν το προαιρετικό Task 2: ο μηχανισμός ανατροφοδότησης υλοποιείται στο `ml_detector/evaluation.py` και το αποτέλεσμα του αποτυπώνεται στο παραπάνω σχήμα.

Συνθήκες και παράμετροι εκτέλεσης

Ωστε να μπορεί οποιοσδήποτε να αναπαραγάγει τα ίδια ακριβώς αποτελέσματα, καταγράφουμε τις συνθήκες εκτέλεσης:

- **Δεδομένα:** η ανάλυση γίνεται εκτός σύνδεσης πάνω σε ένα ενιαίο αρχείο κίνησης (24.110 πακέτα), χωρίς ζωντανή σύλληψη.
- **Επαναληψιμότητα:** σταθερός σπόρος τυχειότητας (1337) στην παραγωγή της κίνησης και στην εκπαίδευση, ώστε κάθε εκτέλεση να δίνει ίδια αποτελέσματα.
- **Μοντέλα:** Random Forest (200 δέντρα, μέγιστο βάθος 8, με εξισορρόπηση κλάσεων), Logistic Regression και Isolation Forest, από τη βιβλιοθήκη `scikit-learn`.

Ολόκληρη η αλυσίδα, από την παραγωγή της κίνησης έως την αξιολόγηση, τρέχει ενιαία μέσω του `run_all.sh` και τεκμηριώνεται στο `README.txt`. Ο πραγματικός Snort τρέχει ξεχωριστά, εκτός σύνδεσης, μόνο για τα στιγμιότυπα των συναγεμών.

Περιορισμοί και απειλές εγκυρότητας

Στο πνεύμα της ειλικρινούς αναφοράς (διάλεξη 7), δηλώνονται ρητά τα όρια εντός των οποίων ισχύουν τα συμπεράσματα:

- **Τι τεκμηριώνεται:** στο σύνολο εξέτασης, ο Snort μόνος πέτυχε ακρίβεια 1.000 και ανάκληση 0.346, το Random Forest $F1=0.916$, και το υβριδικό το καλύτερο $F1=0.926$, με την επισημάνση των παραθύρων ανεξάρτητη από τα χαρακτηριστικά.
- **Συνθετικά δεδομένα:** τα αποτελέσματα προέρχονται από ελεγχόμενη, συνθετικά παραγόμενη κίνηση. Δεν τεκμηριώνεται ότι οι επιδόσεις μεταφέρονται αυτούσιες σε ένα παραγωγικό δίκτυο με διαφορετική κατανομή κίνησης.
- **Εύρος επιθέσεων:** οι μετρικές αφορούν έξι συγκεκριμένους τύπους επιθέσεων και δεν γενικεύονται σε άγνωστες απειλές.
- **Επιλογή παραμέτρων:** τα κατώφλια των κανόνων και τα βάρη της σύντηξης επιλέχθηκαν εμπειρικά και θα χρειαζόνταν επαναρύθμιση σε άλλο περιβάλλον.
- **Μέγεθος παραθύρου:** δουλεύουμε σε παράθυρα του ενός δευτερολέπτου. Επιθέσεις που εκτυλίσσονται σε πολύ μικρότερη ή πολύ μεγαλύτερη κλίμακα χρόνου μπορεί να χρειάζονται διαφορετικό μέγεθος παραθύρου.

Οι περιορισμοί αυτοί δεν αναιρούν το κεντρικό εύρημα, αλλά οριοθετούν με ακρίβεια το πεδίο ισχύος του και υποδεικνύουν συγκεκριμένες κατευθύνσεις για μελλοντική έρευνα.

Συμπέρασμα

Φτιάξαμε ένα ολοκληρωμένο και επαναλήψιμο πλαίσιο για να αξιολογήσουμε ένα υβριδικό σύστημα ανίχνευσης εισβολών, το οποίο συνδυάζει την ανίχνευση βάσει υπογραφών (Snort) με την ανίχνευση βάσει μηχανικής μάθησης. Δεν μείναμε στην επίδειξη ανιχνεύσεων, αλλά αντιμετωπίσαμε το πρόβλημα ως κάτι μετρήσιμο, με πλήρη κύκλο, από τον σχεδιασμό του εργαστηρίου και την παραγωγή της κίνησης, μέχρι τη σύντηξη των δύο επιπέδων και τη συστηματική αξιολόγηση.

Ως προς την αδυναμία των μεμονωμένων προσεγγίσεων, ο Snort έδειξε τέλεια ακρίβεια αλλά χαμηλή ανάκληση (0.346), αφήνοντας να περάσουν οι αργές και οι κρυφές επιθέσεις. Η μηχανική μάθηση ανέβασε πολύ την ανάκληση, αλλά εισήγαγε εσφαλμένους συναγερμούς. Το σημαντικότερο, τα νούμερα δεν βγήκαν τεχνητά τέλεια, ακριβώς επειδή κρατήσαμε την αλήθεια ανεξάρτητη από τα χαρακτηριστικά και αποφεύχθηκε το φαινόμενο της διαρροής δεδομένων.

Ως προς τη λύση, το υβριδικό σύστημα πέτυχε το καλύτερο $F1$ (0.926), ξεπερνώντας κάθε μεμονωμένη προσέγγιση. Οι περιπτώσεις που εξετάσαμε έδειξαν με μετρήσιμο τρόπο ότι τα δύο σκέλη είναι συμπληρωματικά, όχι περιττά. Η μηχανική μάθηση έπιασε 98 παράθυρα που ο Snort

αγνόησε, και ο Snort έπιασε μια κρυφή επίθεση που η μηχανική μάθηση πέρασε για νόμιμη. Οι δύο προαιρετικές επεκτάσεις ενίσχυσαν την εικόνα, δείχνοντας τον ρόλο της ανίχνευσης ανωμαλιών ως μηχανισμού επιτήρησης και τον τρόπο ρύθμισης του σημείου λειτουργίας.

Σταθερός άξονας της εργασίας υπήρξε η ανεξαρτησία της επισήμανσης από τα features, η αναφορά των αστοχιών και η σαφής οριοθέτηση των συμπερασμάτων. Πέρα από την επιβεβαίωση ότι ο υβριδικός συνδυασμός βελτιώνει την ανίχνευση, η εργασία παρέχει μια αυστηρή και επαναλήψιμη μεθοδολογία για την ποσοτικοποίηση αυτής της βελτίωσης, στοιχείο που συνιστά τη βασική της συνεισφορά.

Παραδοτέα και οδηγίες εκτέλεσης

Η υποβολή είναι οργανωμένη σε έναν ενιαίο φάκελο, με σαφή διαχωρισμό ανάμεσα στον κώδικα, στα δεδομένα και στην τεκμηρίωση. Ο κώδικας παραγωγής κίνησης βρίσκεται στον φάκελο **traffic_generation/**, το κομμάτι της μηχανικής μάθησης στον **ml_detector/**, οι κανόνες στον **rules/**, ενώ τα παραγόμενα δεδομένα κατανέμονται στους φακέλους **logs/**, **pcaps/** και **figures/**. Πλήρεις οδηγίες υπάρχουν στα **README.txt**, **TESTBED_DESIGN.md**, **CASE_STUDIES.md** και **SNORT_GUIDE.md**. Ο παρακάτω πίνακας δείχνει ποιο αρχείο υλοποιεί το καθε ζητούμενο:

Task	Υλοποιείται από:
1. Testbed και μοντέλο απειλής	TESTBED_DESIGN.md, traffic_generation/build_dataset.py
2. Παραγωγή και επισήμανση	build_dataset.py, ml_detector/feature_extraction.py
3. Κανόνες Snort	rules/local.rules, rules/snort.lua, ml_detector/snort_emulator.py, rules/snort_kali.lua, rules/rules_kali.rules, logs/alert_fast.txt, screenshots/
4. Μοντέλα ML	feature_extraction.py, train_model.py
5. Hybrid fusion	ml_detector/hybrid_fusion.py
6. Αξιολόγηση	ml_detector/evaluation.py, CASE_STUDIES.md
7. Απόκρυψη και αναστοχασμός	παρούσα αναφορά (ενότητα Task 7)
Προαιρετικό task 1	train_model.py, evaluation.py

Task	Υλοποιείται από:
Προαιρετικό task 2	evaluation.py (feedback loop)

Πέρα από τον κώδικα, η υποβολή περιλαμβάνει την παρούσα αναφορά, τα αρχεία τεκμηρίωσης, τον κατάλογο εξαρτήσεων (requirements.txt) και τον οδηγό εκτέλεσης του Snort για τα στιγμιότυπα.